When Recurrence Wins: Benchmarking LSTM, FNO, and Transformer Models for Fluid Dynamics

Casey Heiskell: 29207982
caseyhei@student.ubc.ca

Ali Salamatian: 32338881
alisalam@student.ubc.ca

Nihar Darbhamulla: 73451312
obashp19@student.ubc.ca

Abstract

Modeling fluid flows remains a major challenge in science and engineering due to the computational cost and complexity of solving the governing Partial Differential Equations (PDEs). Recent advances in scientific machine learning offer data-driven approaches that can learn fluid dynamics in a mesh-free manner, potentially enabling faster simulations. However, there is a lack of systematic benchmarking between traditional and state-of-the-art surrogate models, especially regarding long-range prediction capabilities. In this work, we train three distinct surrogate models—Fourier Neural Operators (FNO), β-VAE + Transformer, and β-VAE + long short-term memory (LSTM)—on PDEBench datasets and a newly curated long-range fluid flow dataset. We extend PDEBench metrics to account for pressure values. We also reimplement the DeepKoopman model using PyTorch to validate earlier findings. Our results show that β-VAE + LSTM outperforms other models, particularly in long-sequence predictions, highlighting the importance of recurrence-based architectures in capturing fluid dynamics over extended time frames. The code is available at: GitHub Repository

1 Introduction

Problems in science and engineering often involve stochastic and deterministic models which are formulated as Partial Differential Equations. Within this scope, one of the most critical problems is that of modeling fluid flow owing to its applications in biology, aerospace and marine systems. While the Navier-Stokes equations can be solved computationally, traditional methods such as spatial discretization and subsequent integration over time are expensive with the added-tradeoff of accuracy. In the context of design-space development and optimization for engineering applications, these solvers pose an often intractable bottleneck due to the complexity of the fluid-flow regime. Data-driven methods can potentially learn the trajectory of these equations from data in a mesh-free manner and can substantially speed-up the simulation process. We aim to apply data-driven methods to capture the trajectories of the equations governing fluid flows, and rigorously evaluate and compare their performance.

Recent advances in scientific machine learning have introduced many new techniques for predicting fluid dynamics. However, there remains a clear need for comprehensive benchmarks that allow fair comparisons between surrogate models. Recently, Takamoto et al. (2024) released a benchmark for time-dependent simulation tasks with clear metrics. The metrics were designed to capture the prediction strength of models in a mean-squared-error sense as well as adherence to fluids first principles through measuring conserved quantities. To our knowledge, no prior work has systematically compared the performance of state-of-the-art (SoTA) surrogate models using these metrics, nor thoroughly evaluated their ability to make long-range predictions.

Course project for UBC CPSC 440/550, April 2025.

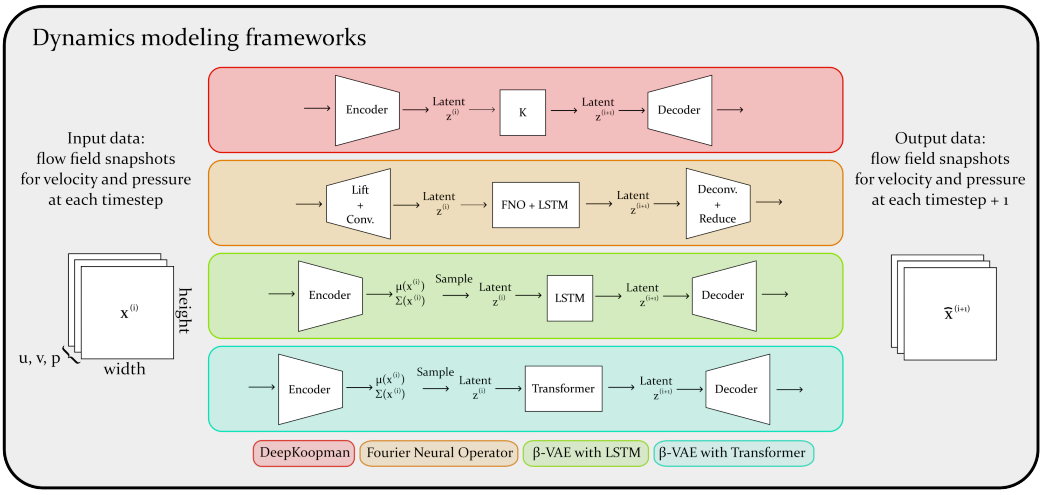

Figure 1: Demonstration of the flow of data from input time to predicted time. It should be noted that some models require a context window of input, rather than simply providing a single $x^{(i)}$ as input.

We seek to train three unique surrogate models on PDEBench datasets as well as our own data for fluid flow over a cylinder. These datasets comprehensively capture different model PDE data, with varying initial conditions, boundary conditions, PDE parameters, and time-series lengths. Additionally, we reimplement the DeepKoopman network using PyTorch and validated some of the previous findings. Further work is needed to fairly compare this approach with the rest of the models on PDEBench.

Our contributions are as follows:

- We train three models—Fourier Neural Operators (FNO), β-VAE + Transformer, and β-VAE + long short-term memory (LSTM)—on PDEBench datasets and extend the benchmark metrics to support datasets that include pressure values.
- We introduce a new dataset featuring longer time-series sequences, train the best-performing models on it, and analyze their performance in long-range prediction tasks.
- We reimplement DeepKoopman in PyTorch and validate key findings from Lusch, Kutz, and Steven L. Brunton (2018).

2 Background and related works

While applying machine learning to fluid flows, two philosophies have prominently been adopted - Reduced-order modeling and Surrogate-models. In the former, the non-linear dynamics is mapped to a latent space to overcome the curse of dimensionality, and within the latent space, computationally cheaper techniques of ODE integration can be applied. In the latter, the underlying physical laws are assumed to be learnable by neural-networks under the umbrella of the universal approximation theorem. In this regard, different approaches have been made towards modeling strategies aimed to capture the underlying dynamics of fluid-flows. We selected a breadth of models from the fluid dynamics literature to test using the PDEBench data and evaluate the proposed metrics. The models include seminal works in surrogate modeling approaches applied within the last 10 years, and in the current study, three classes of models are presented, each of which are described in this section, along with a brief synopsis of the author's findings. The evolution over a single time-step from t to $t + 1$ as modeled by each of the chosen models is illustrated in Fig. 1.

2.1 DeepKoopman

DeepKoopman is a model architecture framework proposed by Lusch, Kutz, and Steven L. Brunton (2018). The model aims to discover a Koopman operator - a theoretical construct which proposes that every non-linear system can be lifted into an infinite-dimensional function space where the evolution

is linear, i.e. there exists a function space $\mathbf{\Phi}$ such that given the non-linear dynamical system

$$\mathbf{x}_{k+1} = \boldsymbol{F}(\mathbf{x}_k), \tag{1}$$

the embedding of $\mathbf{x}$ in $\mathbf{\Phi}$ evolves as

$$\mathbf{\Phi}(\mathbf{x}_{k+1}) = \mathbf{K}\mathbf{\Phi}(\boldsymbol{F}(\mathbf{x}_{k+1})) \tag{2}$$

where $\mathbf{K}$ is a linear-operator in the infinite-dimensional function space (Steven L Brunton et al. 2021).

In the original work, an encoder-propagator-decoder model is trained which learns a lifted latent space and parameters governing temporal evolution. While they illustrated the architecture to work for the case of non-linear dynamical systems with low-degrees of freedom, obtaining a latent space with linearly evolving dynamics with very high-dimensional data is a difficult task. However, the fundamentals of Koopman theory which involve lifting the dynamical system to a higher dimension and searching for operators to evolve the latent-space dynamics remains at the core of modeling high-dimensional dynamical system data.

One of the aims of the current study was to re-develop the original version of DeepKoopman implementation, owing to its archaic setup dating back to the initial days of TensorFlow. However, due to constraints pertaining to model sensitivity, the re-implementation task remained limited in scope towards experiments on the original small-scale problem. Further details about the model can be found in the Appendix.

2.2 Fourier neural operators

Neural operators are discretization free, infinite-dimensional non-linear operators learned through neural networks. In this regard, the neural operator works towards producing a single set of network parameters which are discretization invariant, and adhere to single-shot training. In the context of PDEs, a neural operator can learn a mapping from temporal (Li et al. 2021), forcing (Kovachki et al. 2021), and boundary data to the solution, thereby eliminating knowledge of the PDE. As defined by Li et al. (2021), a representation $v_t \mapsto v_{t+1}$ of functions is defined by iterative updates of the form:

$$v_{t+1}(x) := \sigma\big(W\, v_t(x) + (K(a;\phi)\, v_t)(x)\big), \qquad \forall\, x \in D, \tag{3}$$

where A is the space of inputs and

$$K : A \times \Theta_K \;\longrightarrow\; \mathcal{L}\big(U(D;\mathbb{R}^{d_v}),\, U(D;\mathbb{R}^{d_v})\big)$$

maps to bounded linear operators on $U(D;\mathbb{R}^{d_v})$ and is parameterized by $\phi \in \Theta_K$. $W : \mathbb{R}^{d_v} \to \mathbb{R}^{d_v}$ is a linear transformation, and $\sigma : \mathbb{R} \to \mathbb{R}$ is a non-linear activation function whose action is defined component-wise. The operator K is a kernel integral transformation parameterized by the network. The Fourier Neural operator aims to remove the dependence on $a \in A$ and achieve equivariance. This leads to parameterization of $\kappa_\phi \in K$ in the Fourier space to compute the integral operator efficiently. Li et al. (2021) illustrates the model being capable of achieving zero-shot super resolution in space-time. While this allows for the model to evaluate solutions in highly refined domains, the problem of operator learning is computationally intensive owing to the computation of dense networks within the Fourier Space. With regard to temporal evolution of the dynamics, the Fourier Neural operator can operate in two modes - one as an autoregressive model which allows for its coupling with recurrence capturing networks, and potential expansion towards attention-based mechanisms. A second mode of operation is to evolve the dynamics by treating time as a third dimension in the FNO. The former mode lends itself to long-window predictions, while the latter is limited to the size of window predicted. While FNO is a powerful model, it is very difficult to optimize and computationally intensive to train, and requires some pre-processing layers before operating on multi-channel tensor datasets. Further details are provided in the next section.

2.3 β-Variational autoencoders with latent dynamics predictors

In the current age of machine learning, it is common to turn to either long short-term memory networks (LSTMs) or transformers for sequence prediction tasks. Solera-Rico et al. (2024) propose a method for fluid-flow time series prediction which makes use of latent representations learned via a β-variational autoencoder (β-VAE) to feed next-timestep predictions. This approach is tested in two variants, one with an LSTM and one with a transformer, to evolve the latent system dynamics.

The choice of the β-VAE for representation learning was deliberate. The standard linear approach to reduced order fluid flow modeling, the proper orthogonal decomposition (POD), is favored for its orthogonal latent representations. Orthogonal latent spaces add interpretability, as modes can be compared in isolation and can be related to known physical characteristics of the flow. β-VAEs are capable of learning nearly orthogonal latent spaces built from nonlinear transforms. In this way, the resulting representation can be analyzed in a similar fashion to those of the POD, but with added power in capturing nonlinearities in complicated flows.

The proposed approach effectively entails two stages: a β-VAE is trained on the training data; once it is fit, it encodes the entirety of the training and test dataset. The encoded data is then used to train and test the latent dynamics evolution model, taking the form of an LSTM or transformer. In both cases, the latent dynamics model is given a sequence of time-delayed data and makes next-timestep predictions. These predictions can then be passed through the β-VAE decoder, providing time-advanced predictions in the original data space.

It is shown that both variants are capable of high performance on simple flows. For more complicated dynamics, both variants are also capable of performing well, with significantly smaller latent dimension required compared to POD. It is also noted that for long-range predictions, both models do degrade in accuracy as the prediction horizon increases, especially for chaotic flows.

2.4 PDEBench

With regard to the scientific machine learning community, a unifying dataset with known measures of error and fidelity has been wanting in order to benchmark model performance. Furthermore, when a model is applied on scientific data, its generalizability is subject to performance on metrics pertaining to adherence of conservation laws and boundary conditions in conjunction with response characteristics in the frequency domain. The work of Takamoto et al. (2024) addresses both of these requirements through the PDEBench dataset. The PDEBench dataset is an exhaustive collection of high-fidelity numerical simulation data arising from the solutions to different Partial Differential Equations used to model conservation laws. Of particular interest to us is the incompressible Navier-Stokes dataset which arises from equations governing fluid-flow across a broad range of applications. In addition to the dataset, the study trains SOTA models of its time and evaluates metrics devised to obtain a measure of performance relative to PDE solver. These metrics include cRMSE (also known as CSV error), the RMS error corresponding to a conserved property of the solution, bRMSE, the RMS error corresponding to the boundary conditions applied on the problem, and fRMSE, the RMS error in the fourier space. In the current study, we apply these metrics to each of the models presented in the study, thereby allowing for establishing a benchmark on generalizability of the SOTA models. Samples from the Navier-Stokes dataset are illustrated in Fig. 4

2.5 Our dataset

While the incompressible Navier-Stokes dataset offers a range of initial conditions from which the data is sampled, the length of temporal sequences it offers can be inadequate to model recurrent relationships, which restrain the model's prediction capabilities for long-horizon predictions. In order to address this shortcoming, we constructed a dataset of a canonical flow over a cylinder with a fixed inital condition, varying Reynolds number $Re \in \{100, 150\}$ (a flow parameter governing evolutionary dynamics) and a simulation length consisting of $N_t = 4490$ time samples for each Re. This allows us for the construction of upto $N = 4480$ samples per parameter, and allows for greater data in terms of training networks. The data is obtained on an unstructured grid and downsampled to two sets of spatial resolutions of $(N_\mathrm{x}, N_\mathrm{y}) = (193, 65)$ and $(321, 129)$. A sample illustration is shown in Fig. 5

3 Model processing and training

3.1 Fourier neural operators

In this part of the study, we used the baseline model proposed by Li et al. (2021), and modified it further in order to train with the multi-channel PDEBench Navier-Stokes dataset. The dataset is sequential over a Cartesian grid with 3 channels, and requires lifting the temporal input into a higher-dimensional space for evolving the dynamics. In this case we lift each channel of dimensions

Table 1: Comparison of different methods across PDEBench metrics on Navier-Stokes dataset

Method	RMSE	nRMSE	CSV Error	Max Error
FNO	0.122	0.237	0.482	0.780
β-VAE + Transformer	0.258	0.545	0.00517	1.30
β-VAE + LSTM	0.190	0.409	0.00229	1.03
Method	Boundary RMSE	Fourier 1	Fourier 2	Fourier 3
FNO	0.145	0.0337	–	–
β-VAE + Transformer	0.109	0.0454	0.0230	0.00490
β-VAE + LSTM	0.128	0.0305	0.0174	0.00490

Table 2: Comparison of different methods across PDEBench metrics on Cylinder dataset

Method	RMSE	nRMSE	CSV Error	Max Error
β-VAE + Transformer	0.679	1.123	1.82	2.51
β-VAE + LSTM	0.109	0.189	0.0381	0.482
Method	Boundary RMSE	Fourier 1	Fourier 2	Fourier 3
β-VAE + Transformer	0.236	0.147	0.0564	0.00410
β-VAE + LSTM	0.0495	0.0241	0.0114	0.00140

$(N_\mathrm{x}, N_\mathrm{y}, N_\mathrm{t})$ to $(N_\mathrm{x}, N_\mathrm{y}, D)$. Subsequently, we apply 3-D convolution to capture the correlations between the velocity and pressure channels in space, while also downsampling our grid resolution. 4 dense Fourier Neural Operator layers are applied and shared across the input features. The downsampled data is propagated through 2 RNN-LSTM cells, and then upsampled with transposed convolution to the original dimension. While a substantial body of work exists in the domain of Neural Operators, its use in prediction of multi-dimensional data still remains weakly explored. With a training time of approximately 2 hours for 250 epochs and high patience, the model proved extremely difficult to conduct a hyperparameter search. In order to accomodate for computational constraints, the downsampling convolutional layers had fixed stride and kernel widths, while the best experiments were run with 3 different hyperparameters and the PDEBench dataset with $(128, 128)$ spatial resolution. The results of the metrics evaluated on the best model thus far are presented in Table 1

3.2 β-Variational autoencoders with latent dynamics predictors

We took the baseline models proposed by Solera-Rico et al. (2024) and made minor modifications to allow for training with the PDEBench incompressible Navier-Stokes dataset, as well as our flow over a cylinder data. Namely, we adjusted the dimensions of the input to the encoder and output of the decoder. The data used originally only contained x and y components of the flow velocity. The data used in this study consists of the x and y velocity components, as well as the pressure. Beyond these modifications, we ran a grid-search sweep of a selection of hyper-parameters that we deemed likely to impact performance. We first reserved 100 time-steps for testing, and split the remaining data giving 90% for training and 10% for validation. Our set of hyper parameters include different latent dimension sizes, epochs, β values, embedding dimension and number of attention heads for the easy transformer architecture. You can find more detailed information in appendix A.2.

We compared the performance of each model based on the PDEBench metrics. We select the best performing LSTM and transformer in terms of conserved quantities and frequency content to test further. These models were trained on our long-range time series dataset to analyze the performance for predictions further from initialization. While near-term predictions can be useful, it is often desirable to have a model capable of predicting long time series with maintained accuracy. With complicated dynamics divergence in predictions are common. With this test, we aim to establish if further consideration needs to be given to model capacity in this regard.

β-VAE and LSTM

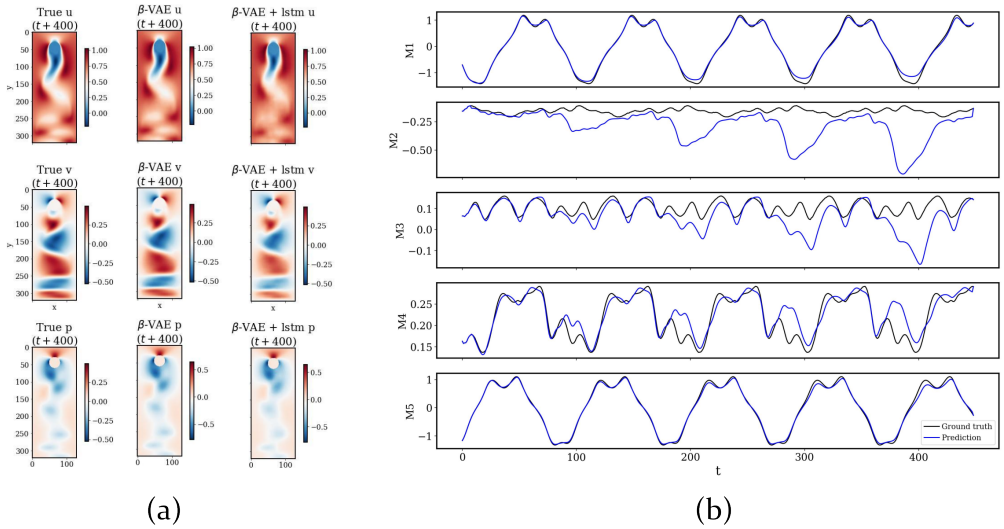

Figure 2: (a) Flow field reconstruction after 400 predicted time-steps using the LSTM. Left shows ground truth, middle shows encoded-decoded ground truth, and right shows the prediction after evolution in the latent space. (b) Time series prediction of the dynamic evolution of the latent variables using the LSTM.

4 Results

We trained and tested β-VAE + Transformer, β-VAE + LSTM, and FNO models on the PDEBench Navier-Stokes incompressible flow dataset. Due to computational constraints, we downsampled the original 512×512 resolution data to 128×128. After hyperparameter tuning, we selected the best-performing models based on validation performance and evaluated them on the test set. As shown in Table 1, all models achieved low RMSE values, indicating strong reconstruction performance. An important complementary metric is the CSV Error, which measures adherence to the continuity condition. Here, the LSTM model outperforms both Transformer and FNO models, achieving less than half the CSV error of the Transformer and over an order of magnitude better performance compared to FNO. To evaluate generalization to longer sequences, we transferred the best hyperparameters from the Navier-Stokes dataset to our dataset modeling flow over a cylinder. Due to resource limitations, no additional hyperparameter tuning was performed. Nevertheless, as shown in Table 2, the LSTM again outperforms the Transformer across all key metrics, including RMSE, CSV Error, and frequency-domain metrics.This could be in part due to our lack of hyper parameter tuning for the transformers model. Moreover, our best-fit models from the Navier-Stokes data use different latent dimension spaces. When we switch the flow characteristics to model our flow over a cylinder dataset, this could have a significant impact on performance. We also pose the possibility, that for dynamics which follow a theoretic ground truth from first principles, the recurrence structure of the LSTM network provides a strong benefit. The flow state at $t + 1$ should be largely dependent on the flow state at t. In this way, LSTMs may have an architectural advantage over the transformer's reliance on attention, which has freedom to ignore recurrence and may learn non-physical dynamics. Perhaps the combination of both recurrency and attention could be advantageous (Deo and Jaiman 2022). Moreover, comparing figures 2 and 3, we can clearly see the superiority of LSTM over the transformer model in both mode matching and reconstructing the flow after 400 time steps.

5 Conclusion and Future work

We presented a comparative study of three surrogate modeling approaches—FNO, β-VAE + Transformer, and β-VAE + LSTM—evaluated on PDEBench and a new long-range fluid flow dataset. By extending PDEBench metrics to include pressure fields and reimplementing the DeepKoopman model, we provided a broader basis for comparison. Our findings highlight the strong performance of the β-VAE + LSTM model, particularly for long-horizon predictions.

Future work will focus on more rigorous hyperparameter optimization, especially for transformer models, and deeper investigation into enhancing the DeepKoopman framework to improve stability and interpretability in modeling complex fluid dynamics.

Acknowledgments

We acknowledge the great help received from our professor and the TA team who would give us an A+. We also want to thank Solera-Rico et al. (2024), as their code provided many useful functions for visualizing results across all models.

References

Steven L Brunton, Marko Budišić, Eurika Kaiser, and J Nathan Kutz (2021). "Modern Koopman theory for dynamical systems." eprint: 2102.12086 (math.DS).

Indu Kant Deo and Rajeev Jaiman (June 2022). "Predicting waves in fluids with deep neural network." *Physics of Fluids* 34.6. ISSN: 1089-7666. DOI: 10.1063/5.0086926. URL: http://dx.doi.org/10.1063/5.0086926.

Nikola B. Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar (2021). "Neural Operator: Learning Maps Between Function Spaces." *CoRR* abs/2108.08481.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar (2021). *Fourier Neural Operator for Parametric Partial Differential Equations*. arXiv: 2010.08895 [cs.LG]. URL: https://arxiv.org/abs/2010.08895.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton (Nov. 2018). "Deep learning for universal linear embeddings of nonlinear dynamics." *Nature Communications* 9.1. ISSN: 2041-1723. DOI: 10.1038/s41467-018-07210-0. URL: http://dx.doi.org/10.1038/s41467-018-07210-0.

Alberto Solera-Rico, Carlos Sanmiguel Vila, Miguel Gómez-López, Yuning Wang, Abdulrahman Almashjary, Scott T M Dawson, and Ricardo Vinuesa (Feb. 2024). "β-Variational autoencoders and transformers for reduced-order modelling of fluid flows." en. *Nat. Commun.* 15.1, p. 1361.

Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Dan MacKinlay, Francesco Alesiani, Dirk Pflüger, and Mathias Niepert (2024). *PDEBENCH: An Extensive Benchmark for Scientific Machine Learning*. arXiv: 2210.07182 [cs.LG]. URL: https://arxiv.org/abs/2210.07182.

A Supplementary material

A.1 DeepKoopman loss function formulation

The original loss function defined by Lusch, Kutz, and Steven L. Brunton (2018) is as follows:

$$\mathcal{L} = \alpha_1(\mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{pred}}) + \mathcal{L}_{\text{lin}} + \alpha_2\mathcal{L}_{\infty} + \alpha_3\|\mathbf{W}\|_2^2 \tag{4}$$

Here α_1, α_2, and α_3 are hyper-parameters. $\mathcal{L}_{\text{recon}}$ is the reconstruction loss that enables the model to learn the latent representation. The prediction loss is defined as the difference between the true state x_{t+1} and the predicted (which is the decoded representation of x_1 transitioned in time with multiplying matrix K).Therefore, this loss teaches the model the linear transformation. L_{lin} enforces the linearity of dynamics and L_{inf}, and the L_2 norm of the weight are the regularizers. You can find the complete formula below, where φ is the encoder and φ^{-1} is the decoder and K represents the

linear transformation through time.

$$\mathcal{L} = \alpha_1(\mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{pred}}) + \mathcal{L}_{\text{lin}} + \alpha_2\mathcal{L}_{\infty} + \alpha_3\|\mathbf{W}\|_2^2$$

$$\mathcal{L}_{\text{recon}} = \left\|\mathbf{x}_1 - \varphi^{-1}\big(\varphi(\mathbf{x}_1)\big)\right\|_{\text{MSE}}$$

$$\mathcal{L}_{\text{pred}} = \frac{1}{S_p}\sum_{m=1}^{S_p}\left\|\mathbf{x}_{m+1} - \varphi^{-1}\big(K^m\varphi(\mathbf{x}_1)\big)\right\|_{\text{MSE}}$$

$$\mathcal{L}_{\text{lin}} = \frac{1}{T-1}\sum_{m=1}^{T-1}\|\varphi(\mathbf{x}_{m+1}) - K^m\varphi(\mathbf{x}_1)\|_{\text{MSE}}$$

$$\mathcal{L}_{\infty} = \left\|\mathbf{x}_1 - \varphi^{-1}\big(\varphi(\mathbf{x}_1)\big)\right\|_{\infty} + \left\|\mathbf{x}_2 - \varphi^{-1}\big(K\varphi(\mathbf{x}_1)\big)\right\|_{\infty}$$

As explained by Lusch, Kutz, and Steven L. Brunton (2018), MSE denotes the mean squared error, and T represents the number of time steps per trajectory. The weights α_1, α_2, and α_3 are hyper-parameters. The integer S_p is a hyperparameter that determines the number of steps considered in the prediction loss.

A.2 β-VAE hyper-parameter tuning space

Here is the complete list of our the hyper parameters we grid searched over:

$$\begin{aligned}
\texttt{latent_dims} &= [2, 3, 5] \\
\texttt{epochs_list} &= [100, 500] \\
\texttt{betas} &= [0.001, 0.005, 0.01] \\
\texttt{embedding dimension} &= [8, 32, 64] \\
\texttt{num_heads} &= [4, 8, 16]
\end{aligned}$$

A.3 FNO hyper-parameter space

The hyperparameters used in the FNO training are

$$\begin{aligned}
\texttt{convolutional_channels_L1} &= [64, 32, 16] \\
\texttt{convolutional_channels_L2} &= [16, 8, 4] \\
\texttt{stride} &= (2, 2, 1) \\
\texttt{epochs} &= 250 \\
\texttt{fully_conn_lift} &= [32, 128]
\end{aligned}$$

A.4 β-VAE and transformer results

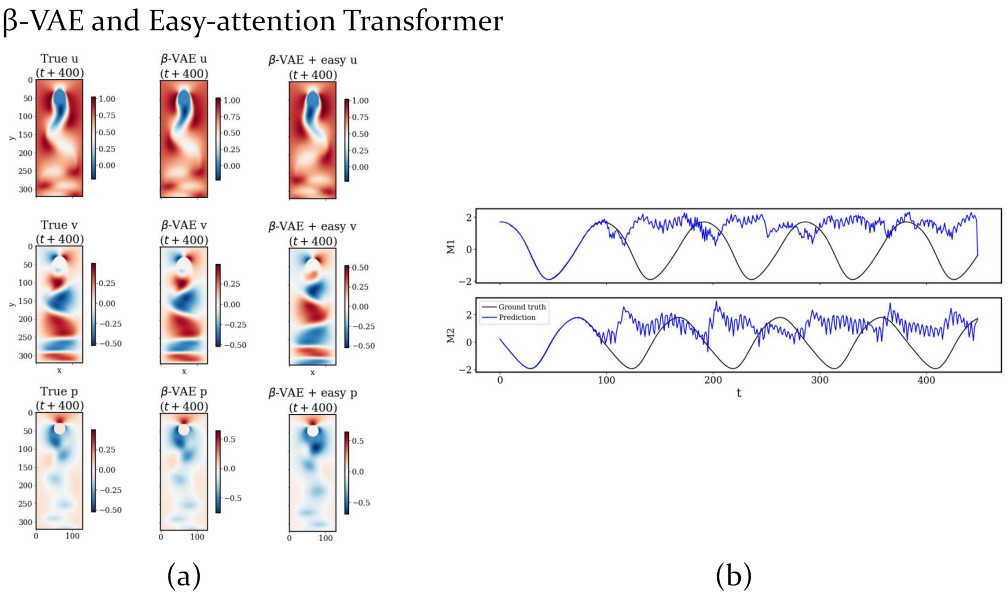

Figure 3: (a) Flow field reconstruction after 400 predicted time-steps using the transformer. Left shows ground truth, middle shows encoded-decoded ground truth, and right shows the prediction after evolution in the latent space. (b) Time series prediction of the dynamic evolution of the latent variables using the transformer.

A.5 Data visualizations

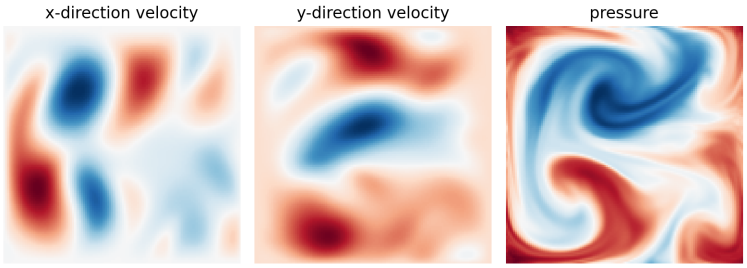

Figure 4: Snapshot of the flow field at the 100th time-step showing all three parameters from PDEBench's incompressible Navier-Stokes dataset.

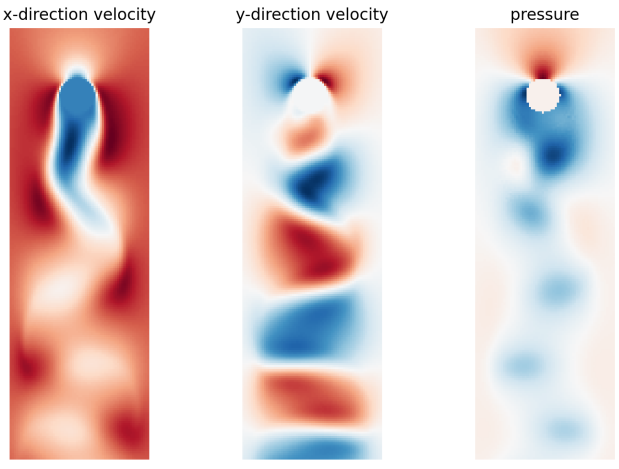

Figure 5: Snapshot of the flow field at the 1000th time-step showing all three parameters from our flow over a cylinder dataset.